Workflow-Oriented Agents

Hybrids, branching, retries, fallbacks

INTRODUCTION

> Where on the deterministic-to-agentic spectrum should each part of the system sit, and how do branching, retries, and fallbacks keep the production path operable when the LLM misfires?

📋 Workflow engine

- A graph of tasks whose structure is known at design time
- Edges, retries, and branch conditions declared up-front
- The engine walks through a DAG; e.g., Airflow, Prefect, Argo
- Predictability: Same inputs, same topology of work

🤖 Pure agent

- An LLM that decides its own next step at runtime
- Structure emerges from the reasoning trace, not from code
- Flexible on open-ended tasks; erodes predictability
- Debugging is harder because the graph is not drawn anywhere

🔀 Hybrid

- A deterministic skeleton with LLM-powered leaves
- Ingest, validate, route, execute, notify
- Classify, extract, summarise; LLM leaves inside the graph
- The production-robust default for most problems

PROPERTIES OF WORKFLOW-ORIENTED SYSTEM

01

Decision Locus

Who decides the next step — us manually at design time, or the model at runtime? The useful axis is not 'workflow vs agent' but where this locus sits.

02

Conditional Branching

Branches route state to the next node based on a closed enum. Static policy, semantic classifier, or cost/latency signal. Each branch is reviewable like an if-statement.

03

Retry Strategy

Every external call chooses one of: immediate retry, exponential backoff with jitter, circuit breaker, or semantic fallback. Retrying blindly is a bug.

04

Fallback Chain

A prioritised list of responders, each progressively cheaper and more reliable. Primary model, backup model, rule-based handler, human queue; the system degrades, but it should not fail.

PROPERTIES OF WORKFLOW-ORIENTED SYSTEM

05

Orchestration Boundary

Some code runs inside the engine; some code runs inside an agent. The boundary is a design choice — and every call across it is a reviewable contract.

06

State Shape

State is a typed object the graph reads and writes. Nodes receive state, return a partial update, and the engine merges. Nothing leaves state that the graph did not name.

07

Idempotency

A retried node must produce the same effect as a single run. LLM calls are not idempotent by default — some providers bill every attempt, some drift at temperature > 0.

08

Event-Driven Triggers

Work starts because something happened: a message arrived, a timer fired, a webhook landed. Event-driven orchestration keeps the engine passive and bounded.

EXAMPLES

System	Deterministic	Branching	Retry/Fallback	Emergent
Airflow DAG	✓	✓	✓	✗
Temporal workflow	✓	✓	✓	✗
LangGraph (declared edges)	✓	✓	✓	~
Hybrid: skeleton + LLM leaves	✓	✓	✓	~
Pure ReAct agent	✗	~	~	✓

NEED FOR A DECISION LOCUS

Predictability compounds

A declared DAG produces the same topology of work every run. Debug once, ship, move on. A pure agent may take three different paths on the same input and each incident is its own investigation.

Hybrid wins on operability

A deterministic skeleton with LLM leaves keeps the workflow auditable, testable, and bounded in cost while still benefiting from the flexibility an LLM offers at classify/extract/summarise leaves.

Failure modes are contained

'The LLM leaf returned junk' is a contained, retryable error. 'The agent went off the rails for 14 minutes and called the payment API 62 times' is not. Containment is an architectural property, not a runtime guard.

SELECTING A BRANCH POINT

A branch is a reviewable contract. When it stops being reviewable, it stops being a branch.

✓ Good branch point

- Closed set of outcomes known at design time.
- Decision function is deterministic — or classifier plus map.
- Each branch has its own SLO and test suite.
- Failure in one branch does not poison the others.
- Observable: the chosen edge and the reason are logged.

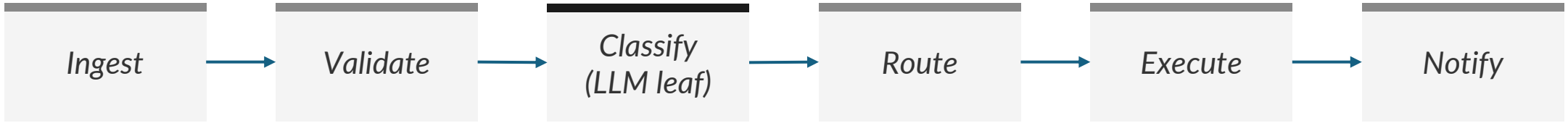
✗ Bad branch point

- Branches are inferred from free-form LLM text.
- No default or fallback branch.
- Branch logic depends on side-effects in the router.
- Router function reaches into external state to decide.
- No telemetry on branch distribution over time.

RETRIES AND FALLBACKS

What a workflow-oriented system should do

- A single global deadline spans the whole call chain
- Retry strategy matched to the failure class, not blanket
- A circuit breaker opens on sustained downstream failure
- A fallback node routes to a cheaper model, then a rule
- Every layer is instrumented; layer mix is monitored



Deterministic nodes

- Ingest: pull the event from the queue
- Validate: schema check, reject on failure
- Route: rule table maps priority to path
- Execute: call the appropriate downstream
- Notify: emit the outcome event

LLM leaf (contained)

- Classify: map free-form input to a closed enum
- Prompt and temperature fixed in code-review
- Output validated against the enum
- Retry and timeout attached to this node only
- Failure here is contained, not a runaway loop

BRANCHING vs ROUTING

Branching

- A decision point whose outcomes are enumerated at design time
- Examples:
  - approve / reject / escalate
  - high / med / low priority
  - *cheap_path* / *expensive_path*
- A branch reads like an if-statement
  Total function over a closed enum

Routing

- A next-node choice that emerges from LLM output at runtime
- Examples:
  - agent picks the next tool to call
  - LLM router over an open intent set
  - free-form next-step planning
- A route may produce classes the designer never anticipated

As a simple rule of thumb for deciding branches:
if you can name the edges up front, it is a branch

RETRY vs FALLBACK

Retry loop

- The same call, repeated with a delay, assumes the error is transient
- Examples:
  - HTTP 429 rate limit, retry with backoff
  - Connection reset, immediate retry once
  - Timeout, exponential backoff with jitter
- Budget: per-call attempts and a wall-clock cap

Fallback chain

- Different calls, in order of preference, assumes the primary path is degraded
- Examples:
  - primary LLM → cheap LLM → rule
  - vector store → keyword index
  - provider A → provider B → human
- Each layer has its own timeout and cost cap

Retries compose multiplicatively. Deadlines compose additively.
Design the deadline first.

Deterministic Orchestration

Linear pipelines, DAGs, parallelism

INTRODUCTION

> When do you reach for a linear pipeline, a DAG, fan-out parallelism, or an agent loop, and why does Airflow belong next to your agent, not underneath it?

Linear pipeline

- A sequence of stages where each consumes the previous output
- No branching; No loops; No parallelism
- The graph is a path
- Reach for it whenever one path fits

DAG

- Vertices and directed edges with no cycle
- Edges encode must-run-before
- Enables parallelism between independent vertices
- Workhorse shape for data engineering

Agent loop

- Cycles with runtime-decided transitions
- Next step is a function of the LLM's output
- Requires explicit termination conditions
- This is not a DAG and should not be forced into one

PROPERTIES OF WORKFLOW-ORIENTED SYSTEM

01

Explicit topology

The graph is declared, not discovered. A reader can trace every possible execution path before running anything.

02

Bounded termination

Execution completes in a finite number of steps by construction. No 'max iterations' escape hatch required.

03

Idempotent stages

Re-running any stage with the same input produces the same output. This is what makes checkpoint-and-resume actually safe.

04

Clean retry semantics

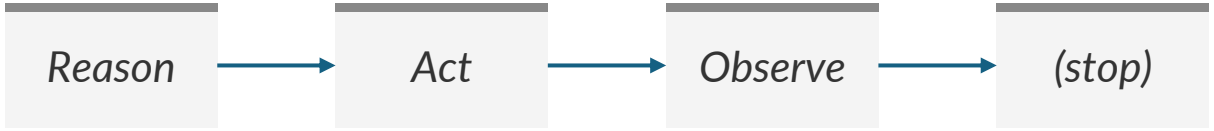
A failed vertex re-runs itself and its downstream — never its upstream. Retries prune cleanly because structure forbids cycles.

NEED FOR PICKING THE RIGHT SHAPE

- **Checkpoint-and-resume is cheap — but only if stages are idempotent**
  - A production pipeline must survive partial failure
  - Persist state per run id after each stage
  - Stages with free-running side effects need guards, dedup keys, or cached results.
- **The critical path determines wall-clock — not fan-out width**
  - For $S \rightarrow (A, B, C) \rightarrow T$, the critical path is $S \rightarrow max(A,B,C) \rightarrow T$
  - Widening fan-out does not slow the run; Lengthening any branch does
  - Optimise the longest branch first
- **Parallel LLM calls have three budgets**
  - A concurrency budget caps in-flight calls; A rate budget respects provider RPS
  - A cost budget stops the fan-out when spend crosses a threshold even if more work is queued

ARCHITECTURE

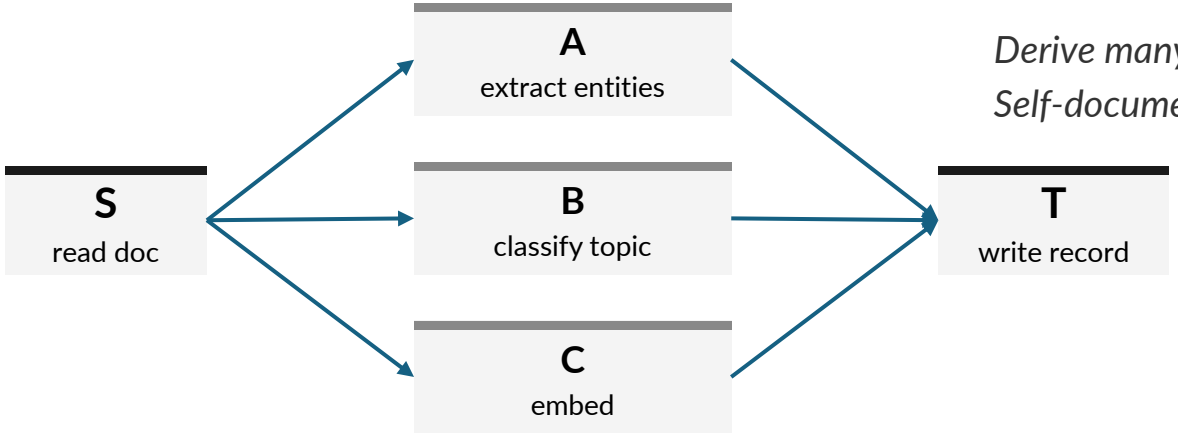
Anti-pattern: agent forced into a static DAG



Pattern: cyclic graph with explicit exit condition



For example



Derive many facets from one input, join at the end
Self-documenting; the graph is the spec

TOPOLOGY vs TRAJECTORY

Topology

- The fixed shape of the graph — nodes and edges declared up front
- Examples:
  - clean → summarize → titlecase
  - S → (A, B, C) → T
  - parsed at DAG build time
- Topology is what you review
  It should be finite and human-readable

Trajectory

- A single concrete run: which edges fired, with what state, in what order
- Examples:
  - *run_id=abc*: S → A → T (skip B, C)
  - *run_id=xyz*: all three branches fired
  - parsed from logs at run time
- Trajectory is what you observe
  One topology produces many trajectories

A deterministic orchestration has one topology
Many trajectories through it

FAN-OUT vs MAP

Fan-out

- Dispatch one input to many parallel workers, no commitment about how outputs combine
- Examples:
  - Send([S1, S2, S3])
  - broadcast to retrievers
  - parallel tool calls
- Fan-out is the spawn step
  Shape without a reducer

Trajectory

- Fan-out plus a declared combine step, a reducer that merges outputs into one state field
- Examples:
  - Annotated[List[str], operator.add]
  - min / max / first-to-finish
  - weighted aggregation
- Map is fan-out with a contract on how results rejoin

GUARDRAILS

Three budgets every parallel LLM workflow must declare:

Concurrency budget

Cap how many in-flight calls can exist at once. Without this, a burst of twenty hits a ten-RPS provider and produces ten answers plus ten “429” errors.

Rate budget

Respect the provider's requests-per-second via a token bucket. Add jittered backoff before you scale fan-out. This is non-optional, not an optimisation.

Cost budget

Stop the fan-out when cumulative spend crosses a threshold, even if more work is queued. The fan-in contract decides if partial results are acceptable.

PITFALLS IN PRODUCTION

Treating the linear pipeline as primitive

A plain function beats a graph when the work is truly linear. The graph framework earns its weight only when branching, parallelism, or persistence enter the picture.

Forgetting partial failure

If nine of ten fan-out calls succeed, decide up front whether the fan-in tolerates missing entries. A deadline-aware all-or-nothing combinator is a deliberate choice, not the default.

Running the agent inside Airflow

Airflow retries are task-local. There is no semantic retry — 'LLM returned junk, retry stricter.' You end up rebuilding LangGraph inside an operator.

Adding a cycle without an exit condition

Cycles are load-bearing for agents, but every loop needs a step budget, a convergence check, or an interrupt. Structure alone will not save you.

Stateful Agents

INTRODUCTION

Where does your agent actually reside?
When does an agent need to remember, who owns the memory - the process, the session store, or the LLM context itself?
And how do you keep that memory bounded across turns, workers, restarts, and long-lived conversations?

Stateful LLM call

- Putting conversation history in the prompt
- The LLM itself is still stateless across calls
- The history lives in your message list, not in the model

Stateful worker

- Holding session data in process memory of a single worker
- Breaks the moment that worker restarts or a load balancer reroutes
- Feels easy, fails in production

Stateful session store

- State persists in external storage keyed by session id
- Any worker can rehydrate it; Survives restarts
- This is the production pattern; the worker is stateless, the store is stateful

PROPERTIES OF STATELESS AGENTS

01

Pure per request

Every request starts with an empty context. The response is produced from the prompt alone. No prior turn carries over.

02

Horizontally trivial

Any worker can serve any request. Scaling is spinning up more replicas. Load balancing has no stickiness requirement.

03

Nothing to lose

A dead worker takes nothing irreplaceable with it. Failure recovery is restart-and-retry, not state reconciliation.

04

Compliance-friendly

No memory means no PII retention by default. For medical, legal, and financial workloads this is a feature, not a gap.

PROPERTIES OF STATEFUL AGENTS

01

Context that persists

Conversation history, preferences, tool results, and intermediate plans survive between requests, scoped to a session or user.

02

Keyed by session

A session_id (or thread_id) identifies whose state to load. The key is the handle that rehydrates the agent.

03

Multi-turn coherent

Follow-up questions, corrections, and resume-after-pause all require state. Stateless products feel broken even when turns are perfect.

04

Infrastructure-heavy

State must be stored, durably written, access-controlled, expired, and sometimes audited. None of this is the LLM's job.

USECASES FOR EACH PATTERN

Criterion	Stateless	Stateful worker	Stateless + store
Single-shot translate / classify / extract	✓	~	~
Multi-turn conversation is the product	✗	~	✓
Horizontal scale across workers	✓	✗	✓
Resume after pause or crash	✗	✗	✓
Compliance forbids memory (e.g. medical)	✓	✗	~
HITL checkpoints in the workflow	✗	~	✓

SESSION MANAGEMENT

Suppose you keep the message history in a Python list on the server:
This is a stateful worker, not a stateful session.

A stateful session will follow the following patterns:

- Workers stay stateless; state lives in a session store
- Every turn loads state by *thread_id* before running the graph
- Every turn writes new state back under the same *thread_id*
- Any worker can serve any turn — restarts are harmless
- Checkpoints (like LangGraph's checkpointer) do this transparently

What the session state carries

messages · tool results · plan · user preferences · counters · checkpoint cursor

ARCHITECTURE

The session lifecycle: **CREATE, READ, UPDATE, CLOSE, EXPIRE**
Every session passes through these five states

CREATE	First interaction. Allocate a session id from a stable business key. Write initial state. Must be idempotent — calling CREATE twice must not corrupt an existing session.
READ	Subsequent turn. Load state by thread_id. Pass it to the graph. LangGraph's checkpointer does this transparently when the thread_id is provided.
UPDATE	After the graph runs. Write new state atomically under the same thread_id. Partial writes corrupt memory in ways that are hard to debug.
CLOSE	Explicit end. User ends the chat, workflow completes, ticket resolves. Mark the session closed so it stops accepting new turns.
EXPIRE	Automatic end. TTL or inactivity timeout fires and reclaims storage. Without this, the session store grows without bound.

PRUNING vs COMPRESSION

Two main ways to manage context window limits.
Prune decides what is lost. Compress decides what survives in reduced form.

Memory pruning

- Decide which messages to keep and which to drop; Nothing is rewritten
- Examples:
  - FIFO: drop oldest first
  - LRU: drop least-referenced
  - Priority: tag at write time
  - Topic: cluster and drop least relevant
- Pruning loses information
  Choose which information

Memory compression

- Replace many old messages with a shorter representation that preserves what matters
- Examples:
  - Rolling summary as a SystemMessage
  - Semantic compaction into a vector store
  - Hybrid: last K verbatim + summary + vector tail
- Compression keeps the past without keeping every word

GUARDRAILS

Session id derivation

Derive *thread_id* from a stable business key — *user_id + conversation_id, ticket_id,* not from a throwaway UUID. Orphaned sessions you cannot resume are worse than no session at all.

Atomic UPDATE writes

Write state back under thread_id atomically. Partial writes are the worst failure mode, the session looks fine but is subtly corrupt. Use the checkpointer's transactional path.

TTL and inactivity expiry

Every session store needs a bounded lifetime. Pick a TTL that matches product behavior (24h for chat, 30d for tickets) and enforce it. The store grows without bound otherwise.

PII retention policy

Checkpoints and summaries both contain raw user data. Treat the checkpoint store as a first-class data asset with documented redaction, deletion-on-request, and GDPR/HIPAA retention rules.

Token budget per turn

Enforce a hard ceiling on in-context history. Prune and compress to fit it. Log a truncation event every time the ceiling fires so silent forgetting becomes visible.

INTEGRATION

App layer	Derive *thread_id* from *user_id + conversation_id*. Hand it to the graph as *config={'configurable': {'thread_id': ...}}*.
Graph layer	LangGraph StateGraph compiled with a checkpointer. The graph reads and writes state transparently on every node boundary.
Checkpointer	MemorySaver for tests, SqliteSaver for single-process durability, PostgresSaver / custom backend for production persistence and resumability.
Session store	External storage — Redis, SQL, or a purpose-built K-V store — keyed by *thread_id*. Indexed on the dimensions you actually query (user, recency, status).
Memory policy	Pruning and compression run BEFORE the LLM call, shaping the in-context history. State still persists in full in the checkpoint store.

PITFALLS

Production pitfalls unique to stateful agents

Summary drift

Re-summarizing from a prior summary accumulates error. By turn 200 the digest describes a conversation that never happened. Re-summarize from the ORIGINAL messages every K turns.

System-message starvation

Aggressive pruning drops the system message along with old turns — the agent forgets its role. Always set include_system=True; tag system messages as highest priority.

Sensitive data in summaries

You pruned raw PII from the history; the summary still contains it. Apply redaction AFTER summarization, not only on raw messages.

Silent truncation

The trim step runs; the user sees no notice; yesterday's context is gone. Log every truncation event; for user-facing products, show a 'continuing from summary' banner.

Cost regression

The summarizer call itself eats two times the tokens you saved. Budget the summarizer; run it only when turn_count % K == 0, not every turn.

Checkpoint bloat

Storing the full transcript on every checkpoint scales quadratically with turn count. Prune and compress BEFORE writing the checkpoint, not only before the LLM call.